

プログラム説明資料

KuboEngine / Octopus

C++20 / DirectX 12で制作した自作ゲームエンジンと、
その上で動作する3Dサバイバルシューティング。

OBB + 空間分割

衝突形状と候補削減

Shadow Map

深度描画とGPU計測

FrameContext / SRV

GPU完了を基準にした寿命管理

CSV / CPU Test

調整値と自動確認

TAKU OKUBO / オオクボ タク

ゲームプログラマー志望

目次

全体設計、衝突判定、描画、リソース管理、検証結果の順に説明します。

01 作品概要

ゲーム内容 / 担当範囲 / 技術構成

02 全体設計

engineとgameの分離 / シーン構成

03 OBB衝突判定

採用理由 / SAT / デバッグ描画

04 候補削減

空間セル / 近傍検索 / CPUテスト

05 Shadow Map

深度描画 / カリング / 調整値

06 リソース寿命

FrameContext / SrvManager / 遷移テスト

07 CSV・検証

採用理由 / 数値検査 / 実行時統計

08 改善履歴と今後

変更した設計 / 次の計測

作品概要

DirectX 12の初期化からゲーム側の処理まで、個人で実装した作品です。



プレイ画面。敵、HUD、ミニマップ、Shadow Mapを表示。

ゲーム内容

敵を倒して経験値を集め、通常弾、周囲弾、雷撃、爆発弾を強化する3Dサバイバルシューティングです。単体、周囲、複数目標、範囲攻撃から強化先を選ぶため、同じ進行でも攻撃構成が変わります。コインはResultシーンで保存し、Titleシーンの恒久強化に使います。

担当範囲

個人制作。DirectX 12の初期化、描画、入力、音声、モデル、パーティクル、シーン管理から、ゲーム側のプレイヤー、敵、武器、HUD、調整用CSV、デバッグ表示まで実装しました。

エンジン層とゲーム層

engine/

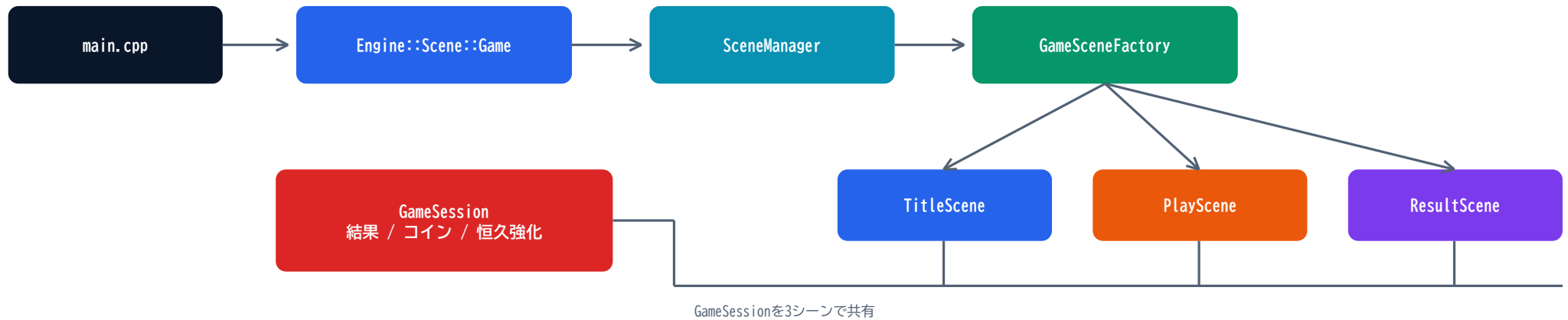
DirectX 12 / FrameContext / Shadow Map / Math / Input / Audio / Model / Particle / Scene

game/directxgame/

Title / Play / Result / Player / Enemy / Weapon / HUD / Effects / Debug / CSV

全体設計

PlaySceneを接続点にし、進行管理、敵、プレイヤーの処理を分けています。



PlaySceneは接続点

PlayScene

プレイヤー、敵、ワールド、HUD、レベルアップUI、ポーズUI、演出、DebugContextを生成して接続します。各機能の中身は専用クラスへ分けています。

GameplayFlowController

Start / Playing / BossIntro / Boss / BossDefeated / Paused / LevelUp / Dead

EnemyManager

敵の生成、EXP、ボス、衝突処理をEnemyCollisionSystemへ接続。

PlayerManager

成長値と武器処理の窓口。PlayerWeaponControllerへ委譲。

main.cpp - シーン生成をFactoryへ渡す

```

auto game = std::make_unique<Engine::Scene::Game>(
    std::make_unique<DirectXGame::GameSceneFactory>(),
    DirectXGame::SceneId::kTitle);
  
```

OBB衝突判定

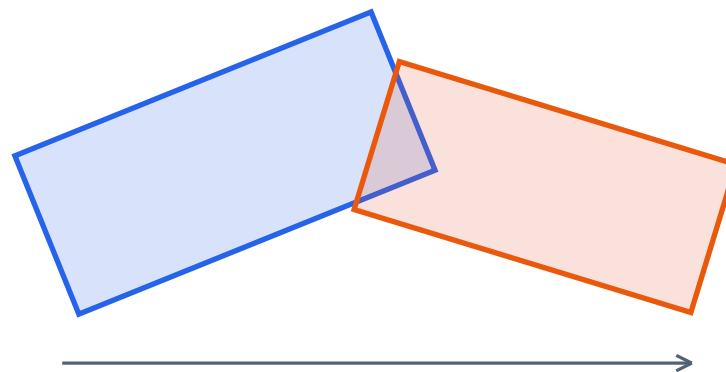
回転する敵や弾で見た目とのずれを減らすため、空間分割後の詳細判定にOBBを使います。

採用理由

球判定は細長い形状で余白が大きくなり、AABBは回転すると見た目より判定範囲が広がります。OBBは向きに沿った判定を作れますが、計算量が増えるため詳細判定に限定しました。

方法	長所	採用理由
球	距離比較だけ	細長い形状では余白が大きい
AABB	単純で軽い	回転時に判定範囲が広がる
OBB	向きに沿う	空間分割後の詳細判定に採用

分離軸判定（SAT）



15本の候補軸へ投影し、1本でも分離すれば非衝突

MyMath.h / MyMath.cpp

```
struct OBB {
    Vector3 center;
    Vector3 orientations[3];
    Vector3 size;
};
constexpr float kEpsilon = 0.0001f;
```

実装上の要点

2つのOBBが持つ3軸ずつと、軸同士の外積9軸を調べます。平行に近い軸同士は浮動小数点誤差の影響を受けやすいため、軸間の回転行列を絶対値化し、各成分へkEpsilonを加えています。

OBB判定の候補削減

OBB判定の前に空間セルを検索し、近くにいる敵だけを判定対象にします。



EnemyCollisionSystem.cpp

```

CollectNearbyEnemies(spatialMap, bulletPosition,
    bulletRadius + kEnemyQueryPadding, nearbyEnemies);
for (Enemy* enemy : nearbyEnemies) {
    if (!MyMath::IsCollision(
        bulletOBB, enemy->GetCollisionOBB())) {
        continue;
    }
    // ■■■■
}

```

実装上の注意

- 中心セルだけでは境界をまたぐ敵を検索できないため、衝突半径が重なる全セルへ登録する。
- 複数セルから同じ敵が入るため、並べ替えて重複を除く。
- 負座標を含むセルキーはuint32_t経由で64bit値へ変換する。
- 通常弾、周囲弾、爆発弾、ドローン弾、敵同士の押し戻しで共通利用する。

敵同士の押し戻しに相当する敵数分と、弾に相当する96回の候補検索を1回の処理単位として比較。

敵数 / 検索回数	全走査 ms	空間分割 ms	結果	候補比較回数
128 / 224	0.0356	0.0417	空間分割 17.2%増	28,672 → 144
256 / 352	0.1137	0.0847	25.5%短縮	90,112 → 428
512 / 608	0.4212	0.2106	50.0%短縮	311,296 → 1,436
1024 / 1120	1.3107	0.3777	71.2%短縮	1,146,880 → 5,108

Release x64 / セル幅 8.0 / 乱数シード固定 / 120サンプル

512体では重複許容0.1696ms、sort/uniqueあり0.2106ms。ただし許容時はヒットが192件から280件へ二重計上されるため、正しさを優先して除外しています。値は広域判定を切り出した測定です。

Shadow Map

ライト視点の深度を先に描き、通常描画で影として参照します。



プレイ画面。敵とプレイヤーの影を地面へ投影。



2048x2048の深度テクスチャを作成し、R32_FLOATのSRVとして通常描画へ渡します。Object3D::DrawShadowはcastsShadowとライト視錐台を確認し、範囲内だけを送信します。

方法	負荷	表現	採用理由
影なし	最小	接地感が弱い	負荷比較用に切替可能
円形影	小	形状と方向を反映しにくい	モデル形状が出ないため不採用
Shadow Map	深度描画を追加	モデル形状と方向を反映	視錐台カリングと統計を追加

PlayScene.cpp

```

if (BeginShadowPass(pos)) {
    player->DrawShadow();
    enemies->DrawShadow();
    EndShadowPass();
}
  
```

2048を選んだ理由

影範囲180x180で1 texel ÷ 0.088。R32は16MiB。1024は4MiBだが輪郭が粗く、4096は64MiBかつ深度ピクセル数が4倍になるため、中間の2048を選びました。

GPU計測 - Debug x64

1980フレーム / フレーム平均 4.1211ms・95%ile 5.3334ms。Shadow Pass平均 0.0781ms・95%ile 0.1004ms / 1.89%。自動遷移時の軽負荷条件で、計測1890回、平均候補1.0体/パス、多数の敵がいる場面は今後PIXで測定します。

DirectX 12のリソース寿命管理

ダブルバッファに合わせてFrameContextを2つ持ち、GPU完了後にリソースを再利用します。

FrameContext x 2 (ダブルバッファ)



DirectXCommon.h

```

struct FrameContext {
    ComPtr<ID3D12CommandAllocator> commandAllocator;
    ComPtr<ID3D12Resource> uploadArena;
    vector<ComPtr<ID3D12Resource>> deferredReleaseResources;
    size_t uploadOffset = 0;
    uint64_t fenceValue = 0;
};
  
```

SrvManager

Freeされたインデックスはすぐ再利用せず、フェンス値と一緒に解放待ちキューへ入れます。GPU完了後に再利用リストへ戻し、使用数、残数、用途を記録します。

SrvManager::Free

```

allocated_[srvIndex] = 0;
--activeCount_;
retiredDescriptors_.push_back({ srvIndex, fenceValue });
  
```

シーン遷移テスト

PASS / 3周 / 1980フレーム / Title・Play・Result = 3・3・3 / SRV最大65。内訳: 2D Texture 43、Cube Texture 1、Structured Buffer 20、Shadow Map 1。

周回終了時のメモリ

Working Set: 341.3 → 339.1 → 339.3MiB。Private Bytes: 468.8 → 466.3 → 466.3MiB。各値は周回終了時に記録しています。

定数バッファはSRVヒープを使わず、FrameContextのアップロード領域からGPU仮想アドレスを渡します。

CSV・デバッグ・検証

調整値をCSVへ分け、読込時の型検査とCPUテストを追加しました。

CSVを選んだ理由

方法	長所	採用理由
C++直書き	型が強い	値変更ごとにビルドが必要
JSON	階層構造に強い	表形式では記述が冗長
独自バイナリ	読込効率を出せる	手編集と差分確認が難しい
CSV	表編集と差分確認	型チェックを追加して採用

実行中に見る値

SoftCapTelemetry / StatisticsDebugPanel

敵数、撃破数、EXP Orb、通常弾、破棄数、パーティクル、FPS履歴、ShadowPassStats、SRV使用数と用途を表示します。

数値の厳密な読込

```
CsvReader::ParseFloat
const float result = std::stof(text, &parsedLength);
if (parsedLength != text.size() || !std::isfinite(result)) {
    throw std::invalid_argument("not a finite float");
}
```

std::stofは1.0xの先頭だけを数値として読み、nanもfloatとして返します。文字列全体を読み切ったか、有限値か、32bit整数の範囲内かを検査し、末尾文字、nan、小数形式の整数、桁あふれをCPUテストへ入れました。

自動確認

CPUテスト / シーン遷移テスト

セルキー、HP/ゲージ範囲、CSV数値変換などをCPUテストで確認。シーン遷移ではフレームごとの値と集計値を別CSVへ記録します。

計測項目: 空間分割のCPU時間 / Shadow GPU時間 / SRV内訳 / 周回別メモリ

改善履歴と今後

制作途中で変更した設計と、次に確認する項目です。

見直した点	現在の形	変更の狙い
シーン生成	GameSceneFactoryへ移動	main.cppを起動処理に限定
ゲーム進行	GameplayFlowController	ポーズやレベルアップの条件を一本化
衝突判定	OBB + 空間分割	見た目とのずれと候補増加を分けて対処
影描画	Shadow Map + カリング + 統計	描画対象とGPU時間を記録
リソース寿命	FrameContext + SrvManager	GPU完了前の再利用を防ぐ
検証	CPUテスト + シーン遷移テスト	手動プレイだけに依存しない

確認済み

- 測定した条件では、空間分割は256体から全走査より短時間
- Shadow Pass平均0.0781ms（自動遷移テスト）
- シーン遷移3周PASS、SRV最大65
- CSV不正値を含むCPUテスト

次に計測する項目

- ゲーム内更新全体の衝突CPU時間
- 多数の敵がいる場面をPIXでも計測
- Play時間と周回数を増やしたメモリ確認
- 起動後に追加したアセットの再読み込み
- リソースIDを型付きハンドルへ統一